

1. AI Assistant

The AI coding assistant I chose was the ChatGPT model GPT 5.5.

2. Programming Task

I chose a CSV data cleaner.

3. AI Conversation Logs

Chat Log 1:

Please create a CSV data cleaner in python.

Thought for a couple of seconds

Here is a simple **CSV data cleaner in Python** that can:

- Remove duplicate rows
- Strip extra spaces from text
- Standardize column names
- Remove completely empty rows and columns
- Fill missing values
- Save a cleaned CSV file

```
import pandas as pd
```

```
from pathlib import Path
```

```
def clean_csv(input_file, output_file=None, fill_missing=""):
```

```
    """
```

Cleans a CSV file and saves the cleaned version.

Parameters:

input_file (str): Path to the CSV file to clean.

output_file (str): Path where the cleaned CSV should be saved.

fill_missing (str): Value used to replace missing cells.

Returns:

pandas.DataFrame: The cleaned DataFrame.

```
    """
```

```
    input_path = Path(input_file)
```

```
    if not input_path.exists():
```

```
        raise FileNotFoundError(f"File not found: {input_file}")
```

```
    if output_file is None:
```

```
        output_file = input_path.with_name(input_path.stem + "_cleaned.csv")
```

```
# Read CSV

df = pd.read_csv(input_file)


# Standardize column names

df.columns = (

    df.columns

    .str.strip()

    .str.lower()

    .str.replace(" ", "_")

    .str.replace("-", "_")

)


# Remove completely empty rows and columns

df = df.dropna(how="all")

df = df.dropna(axis=1, how="all")


# Strip extra spaces from text cells

for column in df.select_dtypes(include="object").columns:

    df[column] = df[column].astype(str).str.strip()


# Replace string versions of missing values

missing_values = ["nan", "None", "NONE", "null", "NULL", "N/A", "n/a", ""]
```

```

df = df.replace(missing_values, pd.NA)

# Fill missing values
df = df.fillna(fill_missing)

# Remove duplicate rows
df = df.drop_duplicates()

# Save cleaned CSV
df.to_csv(output_file, index=False)

print(f"Cleaned CSV saved as: {output_file}")

return df

# Example usage
if __name__ == "__main__":
    clean_csv("data.csv")

```

To use it:

1. Install pandas if you do not have it:

```
pip install pandas
```

2. Put your CSV file in the same folder as the Python script.
3. Rename this line:

```
clean_csv("data.csv")
```

to match your file name, for example:

```
clean_csv("students.csv")
```

It will create a new file called:

```
students_cleaned.csv
```

Chat log 2:

Please create a cvs data cleaner python. It should read an input csv file, clean the data, and save a new cleaned cvs file. It should include removing duplicate rows, trimming extra spaces, standardizing column names, removing empty rows and columns, and handling missing values. Please include comments to explain how the code works so that the code can be easily understood.

Thought for a couple of seconds

Here is a beginner-friendly **CSV data cleaner** in Python. It reads an input CSV file, cleans the data, and saves a new cleaned CSV file.

```
import pandas as pd
```

```
def clean_csv(input_file, output_file):
    """
    Reads a CSV file, cleans the data, and saves the cleaned data
    to a new CSV file.

    Parameters:
        input_file (str): The path to the CSV file you want to clean.
        output_file (str): The path where the cleaned CSV file will be saved.
    """

    # Read the CSV file into a pandas DataFrame
    df = pd.read_csv(input_file)

    # -----
    # 1. Remove duplicate rows
    # -----
    # This removes rows that are exactly the same as another row.
    df = df.drop_duplicates()

    # -----
    # 2. Trim extra spaces from text data
    # -----
    # This removes spaces before and after text in every string cell.
```

```
df = df.map(lambda x: x.strip() if isinstance(x, str) else x)
```

```
# -----
```

```
# 3. Standardize column names
```

```
# -----
```

```
# This makes column names lowercase, removes extra spaces,
```

```
# and replaces spaces with underscores.
```

```
df.columns = (
```

```
    df.columns
```

```
    .str.strip()
```

```
    .str.lower()
```

```
    .str.replace(" ", "_")
```

```
)
```

```
# -----
```

```
# 4. Remove empty rows
```

```
# -----
```

```
# This removes rows where every value is missing.
```

```
df = df.dropna(how="all")
```

```
# -----
```

```
# 5. Remove empty columns
```

```
# -----
```

```

# This removes columns where every value is missing.

df = df.dropna(axis=1, how="all")


# -----

# 6. Handle missing values

# -----

# For numeric columns, fill missing values with the column average.
# For text columns, fill missing values with "Unknown".

for column in df.columns:

    if pd.api.types.is_numeric_dtype(df[column]):

        df[column] = df[column].fillna(df[column].mean())

    else:

        df[column] = df[column].fillna("Unknown")


# -----

# 7. Save the cleaned data to a new CSV file

# -----

# index=False prevents pandas from adding an extra index column.

df.to_csv(output_file, index=False)


print(f'Cleaned CSV file saved as: {output_file}')

```



```
# -----  
  
# Example usage  
  
# -----  
  
# Change these file names to match your own files.  
  
input_csv = "input.csv"  
  
output_csv = "cleaned_output.csv"  
  
  
clean_csv(input_csv, output_csv)
```

To use it:

1. Install pandas if you do not already have it:

```
pip install pandas
```

2. Put your CSV file in the same folder as the Python script.
3. Rename this line to match your file:

```
input_csv = "input.csv"
```

4. Run the script. It will create:

```
cleaned_output.csv
```

Small note: the file type is usually called **CSV**, not **CVS**.

Chat log 3:

Please create a csv data cleaner python. It should read an input csv file, clean the data, and save a new cleaned csv file. It should include removing duplicate rows, trimming extra spaces, standardizing column names, removing empty rows and columns, and handling missing values. Please include comments to explain how the code works so that the code can be easily understood.

Thought for a couple of seconds

Here is a beginner-friendly **CSV data cleaner** in Python. It reads an input CSV file, cleans the data, and saves a new cleaned CSV file.

```
import pandas as pd
```

```
def clean_csv(input_file, output_file):
```

```
    """
```

```
    Reads a CSV file, cleans the data, and saves the cleaned data  
    to a new CSV file.
```

```
    Parameters:
```

```
        input_file (str): The path to the CSV file you want to clean.
```

```
        output_file (str): The path where the cleaned CSV file will be saved.
```

```
    """
```

```

# Read the CSV file into a pandas DataFrame

df = pd.read_csv(input_file)

# -----

# 1. Remove duplicate rows

# -----

# This removes rows that are exactly the same as another row.

df = df.drop_duplicates()

# -----

# 2. Trim extra spaces from text data

# -----

# This removes spaces before and after text in every string cell.

df = df.map(lambda x: x.strip() if isinstance(x, str) else x)

# -----

# 3. Standardize column names

# -----

# This makes column names lowercase, removes extra spaces,
# and replaces spaces with underscores.

df.columns = (
    df.columns
    .str.strip()

```

```
.str.lower()

.str.replace(" ", "_")

)
```

```
# -----

# 4. Remove empty rows

# -----

# This removes rows where every value is missing.

df = df.dropna(how="all")
```

```
# -----

# 5. Remove empty columns

# -----

# This removes columns where every value is missing.

df = df.dropna(axis=1, how="all")
```

```
# -----

# 6. Handle missing values

# -----

# For numeric columns, fill missing values with the column average.

# For text columns, fill missing values with "Unknown".

for column in df.columns:

    if pd.api.types.is_numeric_dtype(df[column]):
```

```

        df[column] = df[column].fillna(df[column].mean())
    else:
        df[column] = df[column].fillna("Unknown")

# -----

# 7. Save the cleaned data to a new CSV file
# -----

# index=False prevents pandas from adding an extra index column.
df.to_csv(output_file, index=False)

print(f'Cleaned CSV file saved as: {output_file}')

# -----

# Example usage
# -----

# Change these file names to match your own files.
input_csv = "input.csv"
output_csv = "cleaned_output.csv"

clean_csv(input_csv, output_csv)

```

To use it:

1. Install pandas if you do not already have it:

```
pip install pandas
```

2. Put your CSV file in the same folder as the Python script.
3. Rename this line to match your file:

```
input_csv = "input.csv"
```

4. Run the script. It will create:

```
cleaned_output.csv
```

Small note: the file type is usually called **CSV**, not **CVS**.

Thought for a second

Here is a version with a simple **Tkinter GUI**. It lets the user choose an input CSV file, choose where to save the cleaned CSV file, and then click a button to clean it.

```
import pandas as pd
```

```
import tkinter as tk
```

```
from tkinter import filedialog, messagebox
```

```
def clean_csv(input_file, output_file):
```

```
    """
```

```
    Reads a CSV file, cleans the data, and saves the cleaned data  
    to a new CSV file.
```

```
    """
```

```
# Read the CSV file into a pandas DataFrame

df = pd.read_csv(input_file)


# Remove duplicate rows

df = df.drop_duplicates()


# Trim extra spaces from text data

df = df.map(lambda x: x.strip() if isinstance(x, str) else x)


# Standardize column names:

# - remove extra spaces

# - make lowercase

# - replace spaces with underscores

df.columns = (

    df.columns

    .str.strip()

    .str.lower()

    .str.replace(" ", "_")

)


# Remove rows where every value is missing

df = df.dropna(how="all")
```

```
# Remove columns where every value is missing
```

```
df = df.dropna(axis=1, how="all")
```

```
# Handle missing values
```

```
# Numeric columns get filled with the column mean.
```

```
# Text columns get filled with "Unknown".
```

```
for column in df.columns:
```

```
    if pd.api.types.is_numeric_dtype(df[column]):
```

```
        df[column] = df[column].fillna(df[column].mean())
```

```
    else:
```

```
        df[column] = df[column].fillna("Unknown")
```

```
# Save the cleaned CSV file
```

```
df.to_csv(output_file, index=False)
```

```
def browse_input_file():
```

```
    """
```

```
    Opens a file picker so the user can select the CSV file to clean.
```

```
    """
```

```
    file_path = filedialog.askopenfilename(
```



```
        title="Select CSV File",  
        filetypes=[("CSV Files", "*.csv")]  
    )
```

```
if file_path:  
    input_file_entry.delete(0, tk.END)  
    input_file_entry.insert(0, file_path)
```

```
def browse_output_file():  
    """  
    Opens a file picker so the user can choose where to save  
    the cleaned CSV file.  
    """
```

```
    file_path = filedialog.asksaveasfilename(  
        title="Save Cleaned CSV File As",  
        defaulttextextension=".csv",  
        filetypes=[("CSV Files", "*.csv")]  
    )
```

```
if file_path:  
    output_file_entry.delete(0, tk.END)
```

```
output_file_entry.insert(0, file_path)
```

```
def run_cleaner():
```

```
    """
```

```
    Gets the input and output file paths from the GUI,  
    runs the cleaner, and shows a success or error message.
```

```
    """
```

```
    input_file = input_file_entry.get()
```

```
    output_file = output_file_entry.get()
```

```
    # Make sure the user selected both files
```

```
    if not input_file:
```

```
        messagebox.showerror("Missing Input File", "Please select an input CSV file.")
```

```
        return
```

```
    if not output_file:
```

```
        messagebox.showerror("Missing Output File", "Please choose where to save the  
        cleaned CSV file.")
```

```
        return
```

```
    try:
```

```
clean_csv(input_file, output_file)

messagebox.showinfo("Success", "CSV file cleaned successfully!")
```

```
except Exception as e:

    messagebox.showerror("Error", f"Something went wrong:\n\n{e}")
```

```
# -----

# GUI setup

# -----
```

```
# Create the main window

root = tk.Tk()

root.title("CSV Data Cleaner")

root.geometry("600x250")
```

```
# Title label

title_label = tk.Label(

    root,

    text="CSV Data Cleaner",

    font=("Arial", 18, "bold")

)

title_label.pack(pady=10)
```

```
# -----  
  
# Input file section  
  
# -----  
  
input_frame = tk.Frame(root)  
input_frame.pack(pady=5, padx=10, fill="x")  
  
input_label = tk.Label(input_frame, text="Input CSV File:")  
input_label.pack(anchor="w")  
  
input_file_entry = tk.Entry(input_frame, width=60)  
input_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)  
  
input_browse_button = tk.Button(  
    input_frame,  
    text="Browse",  
    command=browse_input_file  
)  
input_browse_button.pack(side="right")  
  
# -----  
  
# Output file section
```

```
# -----

output_frame = tk.Frame(root)
output_frame.pack(pady=5, padx=10, fill="x")

output_label = tk.Label(output_frame, text="Save Cleaned CSV As:")
output_label.pack(anchor="w")

output_file_entry = tk.Entry(output_frame, width=60)
output_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)

output_browse_button = tk.Button(
    output_frame,
    text="Browse",
    command=browse_output_file
)
output_browse_button.pack(side="right")

# -----

# Clean button

# -----

clean_button = tk.Button(
```

```
root,

text="Clean CSV",

font=("Arial", 12, "bold"),

command=run_cleaner

)

clean_button.pack(pady=20)


# Start the GUI event loop

root.mainloop()
```

You will still need pandas installed:

```
pip install pandas
```

Since you like running scripts with the VS Code play button, save this as something like:

```
csv_gui_cleaner.py
```

Then press the **Run/Play** button in VS Code. A small window should open where you can choose your CSV file and save the cleaned version.

Please add the option to run the program as usual without the gui

Thought for a second

Below is an updated version that can run **two ways**:

1. **GUI mode** by default
2. **Command-line mode** if you provide input and output file names

```

import pandas as pd

import tkinter as tk

from tkinter import filedialog, messagebox

import argparse


def clean_csv(input_file, output_file):
    """
    Reads a CSV file, cleans the data, and saves the cleaned data
    to a new CSV file.

    Parameters:
        input_file (str): The path to the CSV file you want to clean.
        output_file (str): The path where the cleaned CSV file will be saved.
    """

    # Read the CSV file into a pandas DataFrame
    df = pd.read_csv(input_file)

    # -----

    # 1. Remove duplicate rows
    # -----

    df = df.drop_duplicates()

```

```

# -----

# 2. Trim extra spaces from text data

# -----

# This removes spaces before and after text in every string cell.

df = df.map(lambda x: x.strip() if isinstance(x, str) else x)


# -----

# 3. Standardize column names

# -----

# Example: " First Name " becomes "first_name"

df.columns = (

    df.columns

    .str.strip()

    .str.lower()

    .str.replace(" ", "_")

)


# -----

# 4. Remove empty rows

# -----

# Removes rows where every value is missing.

df = df.dropna(how="all")

```



```
# -----  
  
# 5. Remove empty columns  
  
# -----  
  
# Removes columns where every value is missing.  
df = df.dropna(axis=1, how="all")  
  
  
# -----  
  
# 6. Handle missing values  
  
# -----  
  
# Numeric columns: fill missing values with the column mean.  
# Text columns: fill missing values with "Unknown".  
for column in df.columns:  
    if pd.api.types.is_numeric_dtype(df[column]):  
        df[column] = df[column].fillna(df[column].mean())  
    else:  
        df[column] = df[column].fillna("Unknown")  
  
  
# -----  
  
# 7. Save the cleaned data to a new CSV file  
  
# -----  
  
df.to_csv(output_file, index=False)
```

```
print(f'Cleaned CSV file saved as: {output_file}')
```

```
def browse_input_file():
```

```
    """
```

```
    Opens a file picker so the user can select the CSV file to clean.
```

```
    """
```

```
    file_path = filedialog.askopenfilename(
```

```
        title="Select CSV File",
```

```
        filetypes=[("CSV Files", "*.csv")]
```

```
    )
```

```
    if file_path:
```

```
        input_file_entry.delete(0, tk.END)
```

```
        input_file_entry.insert(0, file_path)
```

```
def browse_output_file():
```

```
    """
```

```
    Opens a file picker so the user can choose where to save  
    the cleaned CSV file.
```

```
    """
```

```
file_path = filedialog.asksaveasfilename(  
    title="Save Cleaned CSV File As",  
    defaultextension=".csv",  
    filetypes=[("CSV Files", "*.csv")]  
)
```

```
if file_path:  
    output_file_entry.delete(0, tk.END)  
    output_file_entry.insert(0, file_path)
```

```
def run_cleaner_from_gui():  
    """  
    Gets the input and output file paths from the GUI,  
    runs the cleaner, and shows a success or error message.  
    """
```

```
input_file = input_file_entry.get()  
output_file = output_file_entry.get()
```

```
if not input_file:  
    messagebox.showerror("Missing Input File", "Please select an input CSV file.")
```

```
    return
```

```
if not output_file:
```

```
    messagebox.showerror("Missing Output File", "Please choose where to save the  
cleaned CSV file.")
```

```
    return
```

```
try:
```

```
    clean_csv(input_file, output_file)
```

```
    messagebox.showinfo("Success", "CSV file cleaned successfully!")
```

```
except Exception as e:
```

```
    messagebox.showerror("Error", f"Something went wrong:\n\n{e}")
```

```
def start_gui():
```

```
    """
```

```
    Creates and starts the GUI window.
```

```
    """
```

```
global input_file_entry
```

```
global output_file_entry
```

```
# Create the main window

root = tk.Tk()

root.title("CSV Data Cleaner")

root.geometry("600x250")


# Title label

title_label = tk.Label(

    root,

    text="CSV Data Cleaner",

    font=("Arial", 18, "bold")

)

title_label.pack(pady=10)


# -----

# Input file section

# -----


input_frame = tk.Frame(root)

input_frame.pack(pady=5, padx=10, fill="x")


input_label = tk.Label(input_frame, text="Input CSV File:")

input_label.pack(anchor="w")
```

```
input_file_entry = tk.Entry(input_frame, width=60)

input_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)


input_browse_button = tk.Button(

    input_frame,

    text="Browse",

    command=browse_input_file

)

input_browse_button.pack(side="right")


# -----

# Output file section

# -----


output_frame = tk.Frame(root)

output_frame.pack(pady=5, padx=10, fill="x")


output_label = tk.Label(output_frame, text="Save Cleaned CSV As:")

output_label.pack(anchor="w")


output_file_entry = tk.Entry(output_frame, width=60)

output_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)
```

```
output_browse_button = tk.Button(

    output_frame,

    text="Browse",

    command=browse_output_file

)

output_browse_button.pack(side="right")


# -----

# Clean button

# -----


clean_button = tk.Button(

    root,

    text="Clean CSV",

    font=("Arial", 12, "bold"),

    command=run_cleaner_from_gui

)

clean_button.pack(pady=20)


# Start the GUI event loop

root.mainloop()
```

```

def main():
    """
    Decides whether to run the program with the GUI
    or from the command line.

    If input and output files are provided, it runs normally
    without opening the GUI.

    If no files are provided, it opens the GUI.
    """

    parser = argparse.ArgumentParser(description="Clean a CSV file.")

    parser.add_argument(
        "input_file",
        nargs="?",
        help="The CSV file to clean."
    )

    parser.add_argument(
        "output_file",
        nargs="?",
        help="The file name for the cleaned CSV."
    )

```


)

```
parser.add_argument(  
    "--gui",  
    action="store_true",  
    help="Open the GUI version of the program."  
)
```

```
args = parser.parse_args()
```

```
# If the user types --gui, always open the GUI.
```

```
if args.gui:
```

```
    start_gui()
```

```
# If the user gives both file names, run without the GUI.
```

```
elif args.input_file and args.output_file:
```

```
    try:
```

```
        clean_csv(args.input_file, args.output_file)
```

```
    except Exception as e:
```

```
        print(f"Error: {e}")
```

```
# If the user gives only one file name, show an error.
```

```
elif args.input_file or args.output_file:
```

```

    print("Error: Please provide both an input file and an output file.")

    print("Example:")

    print("python csv_data_cleaner.py input.csv cleaned_output.csv")


# If the user gives no file names, open the GUI by default.

else:

    start_gui()


# This makes sure main() only runs when this file is run directly.

# It will not automatically run if this file is imported into another file.

if __name__ == "__main__":

    main()

```

You can now run it normally like this:

```
python csv_data_cleaner.py input.csv cleaned_output.csv
```

Or open the GUI like this:

```
python csv_data_cleaner.py --gui
```

Or, since the GUI opens by default, you can simply press the **Run/Play** button in VS Code and the GUI should appear.

please modify it so the gui has the option to go back to running it in the terminal only
as it previously was

Thought for a second

Yes — the cleanest way to do that is to make the GUI include a “**Switch to Terminal Mode**” button.

Since a GUI window cannot literally turn into the VS Code terminal after it is already running, this version does the next best thing: when you click **Switch to Terminal Mode**, it closes the GUI and asks for the input/output file paths in the terminal using `input()`.

Replace your current program with this version:

```
import pandas as pd

import tkinter as tk

from tkinter import filedialog, messagebox

import argparse


def clean_csv(input_file, output_file):
    """
    Reads a CSV file, cleans the data, and saves the cleaned data
    to a new CSV file.

    Parameters:
        input_file (str): The path to the CSV file you want to clean.
        output_file (str): The path where the cleaned CSV file will be saved.
    """
```

```

# Read the CSV file into a pandas DataFrame

df = pd.read_csv(input_file)

# -----

# 1. Remove duplicate rows

# -----

df = df.drop_duplicates()

# -----

# 2. Trim extra spaces from text data

# -----

# This removes spaces before and after text in every string cell.

df = df.map(lambda x: x.strip() if isinstance(x, str) else x)

# -----

# 3. Standardize column names

# -----

# Example: " First Name " becomes "first_name"

df.columns = (

    df.columns

    .str.strip()

    .str.lower()

```

```

        .str.replace(" ", "_", regex=False)
    )

# -----

# 4. Remove empty rows

# -----

# Removes rows where every value is missing.
df = df.dropna(how="all")

# -----

# 5. Remove empty columns

# -----

# Removes columns where every value is missing.
df = df.dropna(axis=1, how="all")

# -----

# 6. Handle missing values

# -----

# Numeric columns: fill missing values with the column mean.
# Text columns: fill missing values with "Unknown".

for column in df.columns:

    if pd.api.types.is_numeric_dtype(df[column]):

        df[column] = df[column].fillna(df[column].mean())

```

else:

df[column] = df[column].fillna("Unknown")

7. Save the cleaned data to a new CSV file

df.to_csv(output_file, index=False)

print(f'Cleaned CSV file saved as: {output_file}')

def run_terminal_mode():

"""

Runs the program in terminal-only mode.

This asks the user to type the input CSV file path
and the output CSV file path directly into the terminal.

"""

print("\nCSV Data Cleaner - Terminal Mode")

print("-----")

input_file = input("Enter the input CSV file path: ")

```
output_file = input("Enter the output cleaned CSV file path: ")
```

```
if not input_file:
```

```
    print("Error: No input file was entered.")
```

```
    return
```

```
if not output_file:
```

```
    print("Error: No output file was entered.")
```

```
    return
```

```
try:
```

```
    clean_csv(input_file, output_file)
```

```
    print("CSV file cleaned successfully!")
```

```
except Exception as e:
```

```
    print(f"Error: Something went wrong:\n{e}")
```

```
def browse_input_file():
```

```
    """
```

```
    Opens a file picker so the user can select the CSV file to clean.
```

```
    """
```

```
file_path = filedialog.askopenfilename(  
    title="Select CSV File",  
    filetypes=[("CSV Files", "*.csv")]  
)
```

```
if file_path:  
    input_file_entry.delete(0, tk.END)  
    input_file_entry.insert(0, file_path)
```

```
def browse_output_file():  
    """  
    Opens a file picker so the user can choose where to save  
    the cleaned CSV file.  
    """
```

```
file_path = filedialog.asksaveasfilename(  
    title="Save Cleaned CSV File As",  
    defaultextension=".csv",  
    filetypes=[("CSV Files", "*.csv")]  
)
```

```
if file_path:
```



```
output_file_entry.delete(0, tk.END)
```

```
output_file_entry.insert(0, file_path)
```

```
def run_cleaner_from_gui():
```

```
    """
```

```
    Gets the input and output file paths from the GUI,
```

```
    runs the cleaner, and shows a success or error message.
```

```
    """
```

```
    input_file = input_file_entry.get()
```

```
    output_file = output_file_entry.get()
```

```
    if not input_file:
```

```
        messagebox.showerror("Missing Input File", "Please select an input CSV file.")
```

```
        return
```

```
    if not output_file:
```

```
        messagebox.showerror(
```

```
            "Missing Output File",
```

```
            "Please choose where to save the cleaned CSV file."
```

```
        )
```

```
        return
```

```
try:

    clean_csv(input_file, output_file)

    messagebox.showinfo("Success", "CSV file cleaned successfully!")


except Exception as e:

    messagebox.showerror("Error", f"Something went wrong:\n\n{e}")
```

```
def switch_to_terminal_mode():

    """

    Closes the GUI and switches the program back to terminal-only mode.

    """

    # Close the GUI window

    root.destroy()

    # After the GUI closes, run the terminal version

    run_terminal_mode()
```

```
def start_gui():

    """
```

Creates and starts the GUI window.

```
"""
```

```
global root
```

```
global input_file_entry
```

```
global output_file_entry
```

```
# Create the main window
```

```
root = tk.Tk()
```

```
root.title("CSV Data Cleaner")
```

```
root.geometry("650x320")
```

```
# Title label
```

```
title_label = tk.Label(
```

```
    root,
```

```
    text="CSV Data Cleaner",
```

```
    font=("Arial", 18, "bold")
```

```
)
```

```
title_label.pack(pady=10)
```

```
# -----
```

```
# Input file section
```

```
# -----
```

```

input_frame = tk.Frame(root)

input_frame.pack(pady=5, padx=10, fill="x")


input_label = tk.Label(input_frame, text="Input CSV File:")

input_label.pack(anchor="w")


input_file_entry = tk.Entry(input_frame, width=60)

input_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)


input_browse_button = tk.Button(
    input_frame,
    text="Browse",
    command=browse_input_file
)

input_browse_button.pack(side="right")


# -----

# Output file section

# -----


output_frame = tk.Frame(root)

output_frame.pack(pady=5, padx=10, fill="x")

```

```
output_label = tk.Label(output_frame, text="Save Cleaned CSV As:")
```

```
output_label.pack(anchor="w")
```

```
output_file_entry = tk.Entry(output_frame, width=60)
```

```
output_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)
```

```
output_browse_button = tk.Button(
```

```
    output_frame,
```

```
    text="Browse",
```

```
    command=browse_output_file
```

```
)
```

```
output_browse_button.pack(side="right")
```

```
# -----
```

```
# Buttons section
```

```
# -----
```

```
button_frame = tk.Frame(root)
```

```
button_frame.pack(pady=20)
```

```
clean_button = tk.Button(
```

```
    button_frame,
```

```
        text="Clean CSV",
        font=("Arial", 12, "bold"),
        command=run_cleaner_from_gui
    )
    clean_button.pack(side="left", padx=10)
```

```
terminal_button = tk.Button(
    button_frame,
    text="Switch to Terminal Mode",
    font=("Arial", 12),
    command=switch_to_terminal_mode
)
terminal_button.pack(side="left", padx=10)
```

```
# Help text
```

```
help_label = tk.Label(
    root,
    text="Terminal Mode lets you type the file paths instead of using the GUI.",
    font=("Arial", 10)
)
help_label.pack(pady=5)
```

```
# Start the GUI event loop
```

```
root.mainloop()
```

```
def main():
```

```
    """
```

```
    Decides whether to run the program with the GUI,  
    terminal input mode, or command-line file arguments.
```

```
    """
```

```
    parser = argparse.ArgumentParser(description="Clean a CSV file.")
```

```
    parser.add_argument(
```

```
        "input_file",
```

```
        nargs="?",
```

```
        help="The CSV file to clean."
```

```
    )
```

```
    parser.add_argument(
```

```
        "output_file",
```

```
        nargs="?",
```

```
        help="The file name for the cleaned CSV."
```

```
    )
```

```
parser.add_argument(
    "--gui",
    action="store_true",
    help="Open the GUI version of the program."
)

parser.add_argument(
    "--terminal",
    action="store_true",
    help="Run in terminal mode and type file paths manually."
)

args = parser.parse_args()

# If the user types --gui, always open the GUI.
if args.gui:
    start_gui()

# If the user types --terminal, run the terminal-only version.
elif args.terminal:
    run_terminal_mode()

# If the user gives both file names, run without the GUI.
```



```

elif args.input_file and args.output_file:

    try:

        clean_csv(args.input_file, args.output_file)

    except Exception as e:

        print(f"Error: {e}")

# If the user gives only one file name, show an error.

elif args.input_file or args.output_file:

    print("Error: Please provide both an input file and an output file.")

    print("Example:")

    print("python csv_data_cleaner.py input.csv cleaned_output.csv")

# If the user gives no file names, open the GUI by default.

else:

    start_gui()

# This makes sure main() only runs when this file is run directly.

# It will not automatically run if this file is imported into another file.

if __name__ == "__main__":

    main()

```

Now you have **three ways** to run it.

GUI mode, which still opens by default:

```
python csv_data_cleaner.py
```

Terminal mode, where it asks you to type the file paths:

```
python csv_data_cleaner.py --terminal
```

Original command-line style:

```
python csv_data_cleaner.py input.csv cleaned_output.csv
```

And inside the GUI, the new **Switch to Terminal Mode** button closes the window and lets you type the paths in the terminal.

Please create a cvs data cleaner python. It should read an input csv file, clean the data, and save a new cleaned cvs file. It should include removing duplicate rows, trimming extra spaces, standardizing column names, removing empty rows and columns, and handling missing values. Please include comments to explain how the code works so that the code can be easily understood.

Thought for a couple of seconds

Here is a beginner-friendly **CSV data cleaner** in Python. It reads an input CSV file, cleans the data, and saves a new cleaned CSV file.

```
import pandas as pd
```

```
def clean_csv(input_file, output_file):
```

```
    """
```

Reads a CSV file, cleans the data, and saves the cleaned data to a new CSV file.

Parameters:

input_file (str): The path to the CSV file you want to clean.

output_file (str): The path where the cleaned CSV file will be saved.

"""

Read the CSV file into a pandas DataFrame

df = pd.read_csv(input_file)

1. Remove duplicate rows

This removes rows that are exactly the same as another row.

df = df.drop_duplicates()

2. Trim extra spaces from text data

This removes spaces before and after text in every string cell.

df = df.map(lambda x: x.strip() if isinstance(x, str) else x)

```
# -----  
  
# 3. Standardize column names  
  
# -----  
  
# This makes column names lowercase, removes extra spaces,  
# and replaces spaces with underscores.  
  
df.columns = (  
    df.columns  
    .str.strip()  
    .str.lower()  
    .str.replace(" ", "_")  
)  
  
# -----  
  
# 4. Remove empty rows  
  
# -----  
  
# This removes rows where every value is missing.  
  
df = df.dropna(how="all")  
  
# -----  
  
# 5. Remove empty columns  
  
# -----  
  
# This removes columns where every value is missing.  
  
df = df.dropna(axis=1, how="all")
```

```

# -----

# 6. Handle missing values

# -----

# For numeric columns, fill missing values with the column average.
# For text columns, fill missing values with "Unknown".

for column in df.columns:

    if pd.api.types.is_numeric_dtype(df[column]):

        df[column] = df[column].fillna(df[column].mean())

    else:

        df[column] = df[column].fillna("Unknown")


# -----

# 7. Save the cleaned data to a new CSV file

# -----

# index=False prevents pandas from adding an extra index column.

df.to_csv(output_file, index=False)

print(f'Cleaned CSV file saved as: {output_file}')


# -----

# Example usage

```

```
# -----
# Change these file names to match your own files.

input_csv = "input.csv"

output_csv = "cleaned_output.csv"


clean_csv(input_csv, output_csv)
```

To use it:

1. Install pandas if you do not already have it:

```
pip install pandas
```

2. Put your CSV file in the same folder as the Python script.
3. Rename this line to match your file:

```
input_csv = "input.csv"
```

4. Run the script. It will create:

```
cleaned_output.csv
```

Small note: the file type is usually called **CSV**, not **CVS**.

Thought for a second

Here is a version with a simple **Tkinter GUI**. It lets the user choose an input CSV file, choose where to save the cleaned CSV file, and then click a button to clean it.

```
import pandas as pd
```

```
import tkinter as tk

from tkinter import filedialog, messagebox


def clean_csv(input_file, output_file):
    """
    Reads a CSV file, cleans the data, and saves the cleaned data
    to a new CSV file.
    """

    # Read the CSV file into a pandas DataFrame
    df = pd.read_csv(input_file)

    # Remove duplicate rows
    df = df.drop_duplicates()

    # Trim extra spaces from text data
    df = df.map(lambda x: x.strip() if isinstance(x, str) else x)

    # Standardize column names:
    # - remove extra spaces
    # - make lowercase
    # - replace spaces with underscores
```

```
df.columns = (  
    df.columns  
    .str.strip()  
    .str.lower()  
    .str.replace(" ", "_")  
)
```

```
# Remove rows where every value is missing
```

```
df = df.dropna(how="all")
```

```
# Remove columns where every value is missing
```

```
df = df.dropna(axis=1, how="all")
```

```
# Handle missing values
```

```
# Numeric columns get filled with the column mean.
```

```
# Text columns get filled with "Unknown".
```

```
for column in df.columns:
```

```
    if pd.api.types.is_numeric_dtype(df[column]):
```

```
        df[column] = df[column].fillna(df[column].mean())
```

```
    else:
```

```
        df[column] = df[column].fillna("Unknown")
```

```
# Save the cleaned CSV file
```



```
df.to_csv(output_file, index=False)
```

```
def browse_input_file():
```

```
    """
```

```
    Opens a file picker so the user can select the CSV file to clean.
```

```
    """
```

```
    file_path = filedialog.askopenfilename(
```

```
        title="Select CSV File",
```

```
        filetypes=[("CSV Files", "*.csv")]
```

```
    )
```

```
    if file_path:
```

```
        input_file_entry.delete(0, tk.END)
```

```
        input_file_entry.insert(0, file_path)
```

```
def browse_output_file():
```

```
    """
```

```
    Opens a file picker so the user can choose where to save  
    the cleaned CSV file.
```

```
    """
```

```
file_path = filedialog.asksaveasfilename(  
    title="Save Cleaned CSV File As",  
    defaultextension=".csv",  
    filetypes=[("CSV Files", "*.csv")]  
)
```

```
if file_path:  
    output_file_entry.delete(0, tk.END)  
    output_file_entry.insert(0, file_path)
```

```
def run_cleaner():  
    """  
    Gets the input and output file paths from the GUI,  
    runs the cleaner, and shows a success or error message.  
    """
```

```
    input_file = input_file_entry.get()  
    output_file = output_file_entry.get()
```

```
    # Make sure the user selected both files  
    if not input_file:
```

```
messagebox.showerror("Missing Input File", "Please select an input CSV file.")  
  
return
```

```
if not output_file:
```

```
    messagebox.showerror("Missing Output File", "Please choose where to save the  
    cleaned CSV file.")  
  
    return
```

```
try:
```

```
    clean_csv(input_file, output_file)  
  
    messagebox.showinfo("Success", "CSV file cleaned successfully!")
```

```
except Exception as e:
```

```
    messagebox.showerror("Error", f"Something went wrong:\n\n{e}")
```

```
# -----
```

```
# GUI setup
```

```
# -----
```

```
# Create the main window
```

```
root = tk.Tk()
```

```
root.title("CSV Data Cleaner")
```

```
root.geometry("600x250")

# Title label

title_label = tk.Label(

    root,

    text="CSV Data Cleaner",

    font=("Arial", 18, "bold")

)

title_label.pack(pady=10)


# -----

# Input file section

# -----


input_frame = tk.Frame(root)

input_frame.pack(pady=5, padx=10, fill="x")


input_label = tk.Label(input_frame, text="Input CSV File:")

input_label.pack(anchor="w")


input_file_entry = tk.Entry(input_frame, width=60)

input_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)
```

```
input_browse_button = tk.Button(  
    input_frame,  
    text="Browse",  
    command=browse_input_file  
)  
input_browse_button.pack(side="right")
```

```
# -----  
# Output file section  
# -----
```

```
output_frame = tk.Frame(root)  
output_frame.pack(pady=5, padx=10, fill="x")
```

```
output_label = tk.Label(output_frame, text="Save Cleaned CSV As:")  
output_label.pack(anchor="w")
```

```
output_file_entry = tk.Entry(output_frame, width=60)  
output_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)
```

```
output_browse_button = tk.Button(  
    output_frame,  
    text="Browse",
```

```

        command=browse_output_file
    )

    output_browse_button.pack(side="right")

# -----

# Clean button

# -----

clean_button = tk.Button(
    root,
    text="Clean CSV",
    font=("Arial", 12, "bold"),
    command=run_cleaner
)

clean_button.pack(pady=20)

# Start the GUI event loop

root.mainloop()

```

You will still need pandas installed:

```
pip install pandas
```

Since you like running scripts with the VS Code play button, save this as something like:

```
csv_gui_cleaner.py
```

Then press the **Run/Play** button in VS Code. A small window should open where you can choose your CSV file and save the cleaned version.

Please add the option to run the program as usual without the gui

Thought for a second

Below is an updated version that can run **two ways**:

1. **GUI mode** by default
2. **Command-line mode** if you provide input and output file names

```
import pandas as pd

import tkinter as tk

from tkinter import filedialog, messagebox

import argparse
```

```
def clean_csv(input_file, output_file):
```

```
    """
```

```
    Reads a CSV file, cleans the data, and saves the cleaned data
    to a new CSV file.
```

```
    Parameters:
```

```
        input_file (str): The path to the CSV file you want to clean.
```

```
        output_file (str): The path where the cleaned CSV file will be saved.
```

```
    """
```

```

# Read the CSV file into a pandas DataFrame

df = pd.read_csv(input_file)


# -----

# 1. Remove duplicate rows

# -----

df = df.drop_duplicates()


# -----

# 2. Trim extra spaces from text data

# -----

# This removes spaces before and after text in every string cell.

df = df.map(lambda x: x.strip() if isinstance(x, str) else x)


# -----

# 3. Standardize column names

# -----

# Example: " First Name " becomes "first_name"

df.columns = (

    df.columns

    .str.strip()

    .str.lower()

```



```

        .str.replace(" ", "_")
    )

# -----

# 4. Remove empty rows

# -----

# Removes rows where every value is missing.
df = df.dropna(how="all")

# -----

# 5. Remove empty columns

# -----

# Removes columns where every value is missing.
df = df.dropna(axis=1, how="all")

# -----

# 6. Handle missing values

# -----

# Numeric columns: fill missing values with the column mean.
# Text columns: fill missing values with "Unknown".
for column in df.columns:
    if pd.api.types.is_numeric_dtype(df[column]):
        df[column] = df[column].fillna(df[column].mean())

```

else:

df[column] = df[column].fillna("Unknown")

7. Save the cleaned data to a new CSV file

df.to_csv(output_file, index=False)

print(f'Cleaned CSV file saved as: {output_file}')

def browse_input_file():

"""

Opens a file picker so the user can select the CSV file to clean.

"""

file_path = filedialog.askopenfilename(

title="Select CSV File",

filetypes=[("CSV Files", "*.csv")]

)

if file_path:

input_file_entry.delete(0, tk.END)

```
input_file_entry.insert(0, file_path)
```

```
def browse_output_file():
```

```
    """
```

```
    Opens a file picker so the user can choose where to save
    the cleaned CSV file.
```

```
    """
```

```
    file_path = filedialog.asksaveasfilename(
        title="Save Cleaned CSV File As",
        defaulttextextension=".csv",
        filetypes=[("CSV Files", "*.csv")]
    )
```

```
    if file_path:
```

```
        output_file_entry.delete(0, tk.END)
```

```
        output_file_entry.insert(0, file_path)
```

```
def run_cleaner_from_gui():
```

```
    """
```

```
    Gets the input and output file paths from the GUI,
```

runs the cleaner, and shows a success or error message.

```
"""
```

```
input_file = input_file_entry.get()
```

```
output_file = output_file_entry.get()
```

```
if not input_file:
```

```
    messagebox.showerror("Missing Input File", "Please select an input CSV file.")
```

```
    return
```

```
if not output_file:
```

```
    messagebox.showerror("Missing Output File", "Please choose where to save the  
cleaned CSV file.")
```

```
    return
```

```
try:
```

```
    clean_csv(input_file, output_file)
```

```
    messagebox.showinfo("Success", "CSV file cleaned successfully!")
```

```
except Exception as e:
```

```
    messagebox.showerror("Error", f"Something went wrong:\n\n{e}")
```

```

def start_gui():
    """
    Creates and starts the GUI window.
    """

    global input_file_entry
    global output_file_entry

    # Create the main window
    root = tk.Tk()

    root.title("CSV Data Cleaner")
    root.geometry("600x250")

    # Title label
    title_label = tk.Label(
        root,
        text="CSV Data Cleaner",
        font=("Arial", 18, "bold")
    )
    title_label.pack(pady=10)

    # -----
    # Input file section

```

```
# -----
```

```
input_frame = tk.Frame(root)
```

```
input_frame.pack(pady=5, padx=10, fill="x")
```

```
input_label = tk.Label(input_frame, text="Input CSV File:")
```

```
input_label.pack(anchor="w")
```

```
input_file_entry = tk.Entry(input_frame, width=60)
```

```
input_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)
```

```
input_browse_button = tk.Button(
```

```
    input_frame,
```

```
    text="Browse",
```

```
    command=browse_input_file
```

```
)
```

```
input_browse_button.pack(side="right")
```

```
# -----
```

```
# Output file section
```

```
# -----
```

```
output_frame = tk.Frame(root)
```

```
output_frame.pack(pady=5, padx=10, fill="x")
```

```
output_label = tk.Label(output_frame, text="Save Cleaned CSV As:")
```

```
output_label.pack(anchor="w")
```

```
output_file_entry = tk.Entry(output_frame, width=60)
```

```
output_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)
```

```
output_browse_button = tk.Button(
```

```
    output_frame,
```

```
    text="Browse",
```

```
    command=browse_output_file
```

```
)
```

```
output_browse_button.pack(side="right")
```

```
# -----
```

```
# Clean button
```

```
# -----
```

```
clean_button = tk.Button(
```

```
    root,
```

```
    text="Clean CSV",
```

```
    font=("Arial", 12, "bold"),
```

```
        command=run_cleaner_from_gui
    )
    clean_button.pack(pady=20)
```

```
# Start the GUI event loop

root.mainloop()
```

```
def main():
```

```
    """
```

```
    Decides whether to run the program with the GUI
    or from the command line.
```

```
    If input and output files are provided, it runs normally
    without opening the GUI.
```

```
    If no files are provided, it opens the GUI.
```

```
    """
```

```
    parser = argparse.ArgumentParser(description="Clean a CSV file.")
```

```
    parser.add_argument(
        "input_file",
```



```
    nargs="?",  
    help="The CSV file to clean."  
)
```

```
parser.add_argument(  
    "output_file",  
    nargs="?",  
    help="The file name for the cleaned CSV."  
)
```

```
parser.add_argument(  
    "--gui",  
    action="store_true",  
    help="Open the GUI version of the program."  
)
```

```
args = parser.parse_args()
```

```
# If the user types --gui, always open the GUI.
```

```
if args.gui:
```

```
    start_gui()
```

```
# If the user gives both file names, run without the GUI.
```

```

elif args.input_file and args.output_file:

    try:

        clean_csv(args.input_file, args.output_file)

    except Exception as e:

        print(f"Error: {e}")

# If the user gives only one file name, show an error.

elif args.input_file or args.output_file:

    print("Error: Please provide both an input file and an output file.")

    print("Example:")

    print("python csv_data_cleaner.py input.csv cleaned_output.csv")

# If the user gives no file names, open the GUI by default.

else:

    start_gui()

# This makes sure main() only runs when this file is run directly.

# It will not automatically run if this file is imported into another file.

if __name__ == "__main__":

    main()

```

You can now run it normally like this:

```
python csv_data_cleaner.py input.csv cleaned_output.csv
```

Or open the GUI like this:

```
python csv_data_cleaner.py --gui
```

Or, since the GUI opens by default, you can simply press the **Run/Play** button in VS Code and the GUI should appear.

please modify it so the gui has the option to go back to running it in the terminal only
as it previously was

Thought for a second

Yes — the cleanest way to do that is to make the GUI include a “**Switch to Terminal Mode**” button.

Since a GUI window cannot literally turn into the VS Code terminal after it is already running, this version does the next best thing: when you click **Switch to Terminal Mode**, it closes the GUI and asks for the input/output file paths in the terminal using `input()`.

Replace your current program with this version:

```
import pandas as pd

import tkinter as tk

from tkinter import filedialog, messagebox

import argparse


def clean_csv(input_file, output_file):
```

```
"""
```

Reads a CSV file, cleans the data, and saves the cleaned data
to a new CSV file.

Parameters:

input_file (str): The path to the CSV file you want to clean.

output_file (str): The path where the cleaned CSV file will be saved.

```
"""
```

```
# Read the CSV file into a pandas DataFrame
```

```
df = pd.read_csv(input_file)
```

```
# -----
```

```
# 1. Remove duplicate rows
```

```
# -----
```

```
df = df.drop_duplicates()
```

```
# -----
```

```
# 2. Trim extra spaces from text data
```

```
# -----
```

```
# This removes spaces before and after text in every string cell.
```

```
df = df.map(lambda x: x.strip() if isinstance(x, str) else x)
```

```
# -----
```

```
# 3. Standardize column names
```

```
# -----
```

```
# Example: " First Name " becomes "first_name"
```

```
df.columns = (  
    df.columns  
    .str.strip()  
    .str.lower()  
    .str.replace(" ", "_", regex=False)  
)
```

```
# -----
```

```
# 4. Remove empty rows
```

```
# -----
```

```
# Removes rows where every value is missing.
```

```
df = df.dropna(how="all")
```

```
# -----
```

```
# 5. Remove empty columns
```

```
# -----
```

```
# Removes columns where every value is missing.
```

```
df = df.dropna(axis=1, how="all")
```

```

# -----

# 6. Handle missing values

# -----

# Numeric columns: fill missing values with the column mean.
# Text columns: fill missing values with "Unknown".

for column in df.columns:

    if pd.api.types.is_numeric_dtype(df[column]):

        df[column] = df[column].fillna(df[column].mean())

    else:

        df[column] = df[column].fillna("Unknown")


# -----

# 7. Save the cleaned data to a new CSV file

# -----

df.to_csv(output_file, index=False)

print(f"Cleaned CSV file saved as: {output_file}")


def run_terminal_mode():

    """

    Runs the program in terminal-only mode.

```

This asks the user to type the input CSV file path
and the output CSV file path directly into the terminal.

```
"""
```

```
print("\nCSV Data Cleaner - Terminal Mode")
```

```
print("-----")
```

```
input_file = input("Enter the input CSV file path: ")
```

```
output_file = input("Enter the output cleaned CSV file path: ")
```

```
if not input_file:
```

```
    print("Error: No input file was entered.")
```

```
    return
```

```
if not output_file:
```

```
    print("Error: No output file was entered.")
```

```
    return
```

```
try:
```

```
    clean_csv(input_file, output_file)
```

```
    print("CSV file cleaned successfully!")
```

```
except Exception as e:
```

```
print(f"Error: Something went wrong:\n{e}")
```

```
def browse_input_file():
```

```
    """
```

```
    Opens a file picker so the user can select the CSV file to clean.
```

```
    """
```

```
    file_path = filedialog.askopenfilename(
```

```
        title="Select CSV File",
```

```
        filetypes=[("CSV Files", "*.csv")]
```

```
    )
```

```
    if file_path:
```

```
        input_file_entry.delete(0, tk.END)
```

```
        input_file_entry.insert(0, file_path)
```

```
def browse_output_file():
```

```
    """
```

```
    Opens a file picker so the user can choose where to save
```

```
    the cleaned CSV file.
```

```
    """
```



```
file_path = filedialog.asksaveasfilename(  
    title="Save Cleaned CSV File As",  
    defaultextension=".csv",  
    filetypes=[("CSV Files", "*.csv")]  
)
```

```
if file_path:  
    output_file_entry.delete(0, tk.END)  
    output_file_entry.insert(0, file_path)
```

```
def run_cleaner_from_gui():  
    """  
    Gets the input and output file paths from the GUI,  
    runs the cleaner, and shows a success or error message.  
    """
```

```
    input_file = input_file_entry.get()  
    output_file = output_file_entry.get()
```

```
    if not input_file:  
        messagebox.showerror("Missing Input File", "Please select an input CSV file.")
```

```
return
```

```
if not output_file:
```

```
    messagebox.showerror(
```

```
        "Missing Output File",
```

```
        "Please choose where to save the cleaned CSV file."
```

```
    )
```

```
return
```

```
try:
```

```
    clean_csv(input_file, output_file)
```

```
    messagebox.showinfo("Success", "CSV file cleaned successfully!")
```

```
except Exception as e:
```

```
    messagebox.showerror("Error", f"Something went wrong:\n\n{e}")
```

```
def switch_to_terminal_mode():
```

```
    """
```

```
    Closes the GUI and switches the program back to terminal-only mode.
```

```
    """
```

```
# Close the GUI window
```

```
root.destroy()
```

```
# After the GUI closes, run the terminal version
```

```
run_terminal_mode()
```

```
def start_gui():
```

```
    """
```

```
    Creates and starts the GUI window.
```

```
    """
```

```
    global root
```

```
    global input_file_entry
```

```
    global output_file_entry
```

```
# Create the main window
```

```
root = tk.Tk()
```

```
root.title("CSV Data Cleaner")
```

```
root.geometry("650x320")
```

```
# Title label
```

```
title_label = tk.Label(
```

```
    root,
```

```
        text="CSV Data Cleaner",
        font=("Arial", 18, "bold")
    )
    title_label.pack(pady=10)

    # -----
    # Input file section
    # -----

    input_frame = tk.Frame(root)
    input_frame.pack(pady=5, padx=10, fill="x")

    input_label = tk.Label(input_frame, text="Input CSV File:")
    input_label.pack(anchor="w")

    input_file_entry = tk.Entry(input_frame, width=60)
    input_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)

    input_browse_button = tk.Button(
        input_frame,
        text="Browse",
        command=browse_input_file
    )
```

```
input_browse_button.pack(side="right")
```

```
# -----
```

```
# Output file section
```

```
# -----
```

```
output_frame = tk.Frame(root)
```

```
output_frame.pack(pady=5, padx=10, fill="x")
```

```
output_label = tk.Label(output_frame, text="Save Cleaned CSV As:")
```

```
output_label.pack(anchor="w")
```

```
output_file_entry = tk.Entry(output_frame, width=60)
```

```
output_file_entry.pack(side="left", padx=(0, 5), fill="x", expand=True)
```

```
output_browse_button = tk.Button(
```

```
    output_frame,
```

```
    text="Browse",
```

```
    command=browse_output_file
```

```
)
```

```
output_browse_button.pack(side="right")
```

```
# -----
```

```
# Buttons section
```

```
# -----
```

```
button_frame = tk.Frame(root)
```

```
button_frame.pack(pady=20)
```

```
clean_button = tk.Button(  
    button_frame,  
    text="Clean CSV",  
    font=("Arial", 12, "bold"),  
    command=run_cleaner_from_gui  
)
```

```
clean_button.pack(side="left", padx=10)
```

```
terminal_button = tk.Button(  
    button_frame,  
    text="Switch to Terminal Mode",  
    font=("Arial", 12),  
    command=switch_to_terminal_mode  
)
```

```
terminal_button.pack(side="left", padx=10)
```

```
# Help text
```

```
help_label = tk.Label(
    root,
    text="Terminal Mode lets you type the file paths instead of using the GUI.",
    font=("Arial", 10)
)
help_label.pack(pady=5)
```

```
# Start the GUI event loop
root.mainloop()
```

```
def main():
    """
    Decides whether to run the program with the GUI,
    terminal input mode, or command-line file arguments.
    """
```

```
parser = argparse.ArgumentParser(description="Clean a CSV file.")
```

```
parser.add_argument(
    "input_file",
    nargs="?",
    help="The CSV file to clean."
```

)

```
parser.add_argument(  
    "output_file",  
    nargs="?",  
    help="The file name for the cleaned CSV."
```

)

```
parser.add_argument(  
    "--gui",  
    action="store_true",  
    help="Open the GUI version of the program."
```

)

```
parser.add_argument(  
    "--terminal",  
    action="store_true",  
    help="Run in terminal mode and type file paths manually."
```

)

```
args = parser.parse_args()
```

```
# If the user types --gui, always open the GUI.
```



```
if args.gui:

    start_gui()

# If the user types --terminal, run the terminal-only version.

elif args.terminal:

    run_terminal_mode()

# If the user gives both file names, run without the GUI.

elif args.input_file and args.output_file:

    try:

        clean_csv(args.input_file, args.output_file)

    except Exception as e:

        print(f"Error: {e}")

# If the user gives only one file name, show an error.

elif args.input_file or args.output_file:

    print("Error: Please provide both an input file and an output file.")

    print("Example:")

    print("python csv_data_cleaner.py input.csv cleaned_output.csv")

# If the user gives no file names, open the GUI by default.

else:

    start_gui()
```

```
# This makes sure main() only runs when this file is run directly.  
  
# It will not automatically run if this file is imported into another file.  
  
if __name__ == "__main__":  
    main()
```

Now you have **three ways** to run it.

GUI mode, which still opens by default:

```
python csv_data_cleaner.py
```

Terminal mode, where it asks you to type the file paths:

```
python csv_data_cleaner.py --terminal
```

Original command-line style:

```
python csv_data_cleaner.py input.csv cleaned_output.csv
```

And inside the GUI, the new **Switch to Terminal Mode** button closes the window and lets you type the paths in the terminal.

Tests:

Program 1:

None of the tests passed due to ChatGPT generating an incomplete program.

Program 2:

PASS: csvBasicTest.csv
PASS: csvDuplicatesTest.csv
PASS: csvMissingInfoTest.csv
PASS: csvEmptyRowsColsTest.csv
FAIL: csvInconsistantTest.csv
Column mismatch.
Expected: ['first_name', 'last_name', 'student_id', 'major']
Actual: ['first_name', 'last-name', 'student_id', 'major']

Program 3:

PASS: csvBasicTest.csv
PASS: csvDuplicatesTest.csv
FAIL: csvMissingInfoTest.csv
Column mismatch.
Expected: ['name', 'age', 'city', 'major']
Actual: ['name', 'age', 'city']
FAIL: csvEmptyRowsColsTest.csv
Column mismatch.
Expected: ['name', 'age', 'city']
Actual: ['first_name', 'last-name', 'student_id', 'major']
FAIL: csvInconsistantTest.csv
Column mismatch.
Expected: ['first_name', 'last_name', 'student_id', 'major']
Actual: ['name', 'age', 'city', 'major']

Results:

The first program ChatGPT generated failed to run properly, so it failed all of the test cases. All the tests for the second program generated by ChatGPT passed, except for csvInconsistentTest. This test tested column names with inconsistent naming conventions. This program had an issue converting “last-name” to the standardized “last_name.” The third program generated by ChatGPT passed the first two tests but failed the other three, including the edge cases. I had ChatGPT generate a GUI for this program, which created an issue when running the tests, where the tests had to be manually input into the GUI and then evaluated by the testing program. The problem this program seems to have with the tests is that it incorrectly saves the output file, as the output sometimes matches the expected response of another test.

It is evident that the deviation in quality of the AI’s output can be directly linked with how specific the prompt it is given. The prompt it was given for the first program was a single sentence and, as a result, the program it generated was incomplete. The prompt for the second program was highly specific and resulted in a program that was much more complete and passed nearly all of my tests. The third

program was an outlier in this pattern, but I believe it can be explained. I built off my prompt for the second program by asking it to create a GUI. The GUI made my tests more difficult to run and, as the AI chatbot focused on the generation of a GUI, the other elements of the program fell to the wayside and resulted in a program that did not accomplish its core goal.

Going forward, the next time I prompt an AI chatbot for code, I will remember to be highly specific in my request in order to get code that accomplishes my chosen task successfully.